

torch.fft.fftshift#

<https://pytorch.org/docs/stable/generated/torch.fft.fftshift.html>

Description:

Reorders n-dimensional FFT data, as provided by `fftn()`, to have negative frequency terms first. This performs a periodic shift of n-dimensional data such that the origin (0, ..., 0) is moved to the center of the tensor. Specifically, to `input.shape[dim] // 2` in each selected dimension. By convention, the FFT returns positive frequency terms first, followed by the negative frequencies in reverse order, so that `f[-i]` for all `i`

Function Signature

```
torch.fft.fftshift(input, dim=None) → Tensor#
```

Parameters

Code Examples

```
>>> f = torch.fft.fftfreq(4)
>>> f
tensor([ 0.0000,  0.2500, -0.5000, -0.2500])
```

```
>>> torch.fft.fftshift(f)
tensor([-0.5000, -0.2500,  0.0000,  0.2500])
```

```
>>> x = torch.fft.fftfreq(5, d=1/5) + 0.1 *
torch.fft.fftfreq(5, d=1/5).unsqueeze(1)
>>> x
tensor([[ 0.0000,  1.0000,  2.0000, -2.0000, -1.0000],
        [ 0.1000,  1.1000,  2.1000, -1.9000, -0.9000],
        [ 0.2000,  1.2000,  2.2000, -1.8000, -0.8000],
        [-0.2000,  0.8000,  1.8000, -2.2000, -1.2000],
        [-0.1000,  0.9000,  1.9000, -2.1000, -1.1000]])
```

```
>>> torch.fft.fftshift(x)
tensor([[-2.2000, -1.2000, -0.2000,  0.8000,  1.8000],
        [-2.1000, -1.1000, -0.1000,  0.9000,  1.9000],
        [-2.0000, -1.0000,  0.0000,  1.0000,  2.0000],
        [-1.9000, -0.9000,  0.1000,  1.1000,  2.1000],
        [-1.8000, -0.8000,  0.2000,  1.2000,  2.2000]])
```

```
>>> x_centered = torch.arange(-5, 5)
>>> x_uncentered = torch.fft.ifftshift(x_centered)
>>> fft_uncentered = torch.fft.fft(x_uncentered)
```

```
>>> fft_centered = torch.fft.fftshift(fft_uncentered)
```

```
>>> x_centered_2 =  
torch.fft.fftshift(torch.fft.ifft(torch.fft.ifftshift(fft_  
centered)))  
>>>  
torch.testing.assert_close(x_centered.to(torch.complex64),  
x_centered_2, check_stride=False)
```

Notes & Warnings

NOTE

By convention, the FFT returns positive frequency terms first, followed by the negative frequencies in reverse order, so that $f[-i]$ for all $0 < i \leq n/2$ in Python gives the negative frequency terms. `fftshift()` rearranges all frequencies into ascending order from negative to positive with the zero-frequency term in the center.

NOTE

For even lengths, the Nyquist frequency at $f[n/2]$ can be thought of as either negative or positive. `fftshift()` always puts the Nyquist term at the 0-index. This is the same convention used by `fttfreq()`.

Detailed Documentation

Documentation

Reorders n-dimensional FFT data, as provided by `fftn()`, to have negative frequency terms first.

This performs a periodic shift of n-dimensional data such that the origin (0, ..., 0) is moved to the center of the tensor. Specifically, to `input.shape[dim] // 2` in each selected dimension.

By convention, the FFT returns positive frequency terms first, followed by the negative frequencies in reverse order, so that $f[-i]$ for all $0 < i \leq n/2 < i \leq n/2$ in Python gives the negative frequency terms. `fftshift()` rearranges all frequencies into ascending order from negative to positive with the zero-frequency term in the center.

For even lengths, the Nyquist frequency at $f[n/2]$ can be thought of as either negative or positive. `fftshift()` always puts the Nyquist term at the 0-index. This is the same convention used by `fftfreq()`.

`input` (Tensor) – the tensor in FFT order

`dim` (int, Tuple[int], optional) – The dimensions to rearrange. Only dimensions specified here will be rearranged, any other dimensions will be left in their original order. Default: All dimensions of input.

Also notice that the Nyquist frequency term at $f[2]$ was moved to the beginning of the tensor.

This also works for multi-dimensional transforms:

`fftshift()` can also be useful for spatial data. If our data is defined on a centered grid $([-(N//2), (N-1)//2])$ then we can use the standard FFT defined on an uncentered grid $([0, N])$ by first applying an `ifftshift()`.

Similarly, we can convert the frequency domain components to centered convention by applying `fftshift()`.

The inverse transform, from centered Fourier space back to centered spatial data, can be performed by applying the inverse shifts in reverse order: